

Módulo 3 - SQL com SQLite (Adaptado para SQLiteOnline)

Projeto: Data Analytics com Dados Abertos da PRF

Base: Acidentes 2025 agrupados por ocorrência

1. Verificar a versão do SQLite para garantir compatibilidade
2. Exibir a estrutura (colunas e tipos) da tabela importada 'acidentes_prf_2025'
3. Contar o número total de registros/ocorrências da base
4. Excluir a view base se ela já existir no banco para evitar conflito
5. Criar a view base com a flag 'acidente_fatal' (1 para mortos ≥ 1 , senão 0)
6. Calcular métricas gerais: total de acidentes, total de fatais e o % de letalidade
7. Agregar acidentes, mortos e % de fatais por Estado (UF), filtrando os com ao menos 100 casos
8. Listar as 30 rodovias (BRs) mais letais em número absoluto de mortos
9. Agrupar a evolução temporal dos acidentes por Ano e Mês (extraídos da data)
10. Analisar a relação bivariada entre o Tipo de Acidente e o % de ocorrências fatais
11. Analisar as 30 Principais Causas de Acidentes ordenadas pela maior taxa de letalidade
12. Comparar a gravidade dos acidentes de acordo com a Fase do Dia (noite, pleno dia, etc.)
13. Avaliar a influência da Condição Meteorológica no % de acidentes fatais
14. Comparar a letalidade do acidente de acordo com o Tipo de Pista (simples, dupla, múltipla)

15. Analisar a combinação de dois fatores (Pista + Fase do Dia) e a cobertura em relação ao total
16. Calcular o efeito 'Lift' (razão entre a taxa de letalidade do tipo e a taxa média geral)
17. Criar a view 'vw_indicadores_mensais' para facilitar relatórios temporais
18. Criar a view 'vw_indicadores_uf_br' consolidadas por localização para uso em Dashboards.

Arquivos que deverão ser entregues:

1) Script SQL comentado.

→ Um arquivo .sql com consultas documentadas, reexecutável por outro aluno ou avaliador;

2) Resultados em CSV.

→ Consultas por UF, BR, mês, causa, tipo, clima e fase do dia exportadas para resultados/;

3) Consultas Bivariadas.

→ Análises cruzando variáveis explicativas com acidente_fatal;

4) Visão Agregada.

→ Uma view consolidada pronta para alimentar o dashboard;

Comandos Extras SQL (Análise PRF - SQLiteOnline)

PÁGINA 1: Inspeção, Criação de Visões e Agregações Básicas

1. Comandos de Inspeção e Ambiente

Antes de analisar os dados, usamos comandos para checar a versão e a estrutura da tabela importada no SQLiteOnline:

Comando / Sintaxe Exemplo	O que faz no Script?
<code>SELECT sqlite_version();</code>	Retorna a versão exata do motor SQLite rodando no navegador.
<code>PRAGMA table_info(tabela);</code>	Exibe o 'dicionário' da tabela: nome de todas as colunas, tipos de dados e se aceitam valores nulos.

2. Estruturação da Análise (DROP VIEW & CREATE VIEW)

Para evitar retrabalho, criamos Views (tabelas virtuais). O SQLiteOnline exige remover a view anterior antes de recriá-la, pois não aceita CREATE OR REPLACE:

```
-- 1º: Remove a view antiga se ela já existir para não dar erro de duplicidade
DROP VIEW IF EXISTS vw_acidentes_base;

-- 2º: Cria a view adicionando a regra de negócio do projeto (acidente_fatal = 1 ou 0)
CREATE VIEW vw_acidentes_base AS
SELECT
    *,
    CASE
        WHEN CAST(mortos AS INTEGER) >= 1 THEN 1
        ELSE 0
    END AS acidente_fatal
FROM acidentes_prf_2025;
```

Conceitos-Chave aplicados acima

`CAST(coluna AS INTEGER)`: Converte texto armazenado no CSV original para número inteiro, permitindo cálculos matemáticos.

`CASE WHEN ... THEN ... ELSE ... END`: Lógica condicional (equivalente ao IF da programação) para criar a flag de letalidade.

3. Agregações e Métricas Primárias (SELECT, SUM, COUNT, ROUND)

```
SELECT
    COUNT(*) AS total_acidentes,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base;
```

Conceitos-Chave aplicados acima

COUNT(*): Conta o número total de linhas/registros.

SUM(...): Soma o valor numérico dos elementos (como a flag é 1 para fatal e 0 para não fatal, a soma dá o total de acidentes fatais).

ROUND(valor, 2): Arredonda o resultado numérico para 2 casas decimais.

Multiplicação por 100.0: O .0 força o SQLite a realizar uma divisão decimal em vez de inteira.

PÁGINA 2: Agrupamentos, Filtros de Grupo e Análises Avançadas

4. Agrupamentos com Filtro de Corte (GROUP BY, HAVING, ORDER BY)

```
SELECT
    uf,
    COUNT(*) AS total_acidentes,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base
GROUP BY uf
HAVING COUNT(*) >= 100
ORDER BY perc_fatais DESC;
```

Conceitos-Chave aplicados acima

GROUP BY coluna: Agrupa todas as linhas que possuem o mesmo valor naquela coluna (ex: junta todos os registros do estado 'PE').

HAVING COUNT(*) >= 100: Filtra os grupos após a agregação. Garante relevância estatística, mantendo apenas categorias com 100 ou mais acidentes.

ORDER BY coluna DESC: Ordena o resultado do maior para o menor (decrescente).

5. Manipulação de Datas (strftime)

```
SELECT
    strftime('%Y', data_inversa) AS ano,
    strftime('%m', data_inversa) AS mes,
    COUNT(*) AS total_acidentes
FROM vw_acidentes_base
GROUP BY ano, mes;
```

📌 Máscaras de Data no SQLite: %Y extrai o Ano com 4 dígitos e %m extrai o Mês com 2 dígitos.

6. Análises Bivariadas e Métricas Avançadas (WITH / CTE)

```
-- Cria uma tabela temporária 'global' com a taxa geral de acidentes
WITH global AS (
    SELECT
        1.0 * SUM(acidente_fatal) / COUNT(*) AS taxa_global,
        COUNT(*) AS total_geral
    FROM vw_acidentes_base
)
-- Usa a tabela temporária no SELECT principal para calcular a métrica de LIFT
SELECT
    b.tipo_acidente,
    COUNT(*) AS total_acidentes,
    ROUND((1.0 * SUM(b.acidente_fatal) / COUNT(*)) / g.taxa_global, 2) AS lift
FROM vw_acidentes_base b, global g
GROUP BY b.tipo_acidente, g.taxa_global;
```

💡 Conceitos-Chave aplicados acima

WITH nome AS (...): Permite criar uma 'subconsulta organizada' no topo do código para reuso.

Cálculo de Lift: Avalia quantas vezes um determinado fator (ex: Colisão Frontal) aumenta a chance de fatalidade em comparação à média nacional.

SQL com SQLiteOnline

Aplicado à Ciência de Dados

Comandos SQL comentados e explicados

Estudo de caso: acidentes rodoviários da Polícia Rodoviária Federal

Volume 1 — Material técnico para SQLiteOnline

Apresentação

Este documento reúne, organiza e explica os principais comandos SQL desenvolvidos para a análise de acidentes rodoviários. O script original foi escrito para DuckDB. Nesta versão, os exemplos foram adaptados para o SQLiteOnline, preservando a finalidade analítica e substituindo recursos que não fazem parte do SQLite.

Importante: No SQLiteOnline, a importação e a exportação de arquivos CSV são feitas pela interface do site. Comandos como `read_csv_auto`, `COPY` e `CREATE OR REPLACE` não pertencem ao SQLite padrão e, por isso, são substituídos ou explicados separadamente.

Como usar este material

- Importe previamente o arquivo CSV no SQLiteOnline e nomeie a tabela como `acidentes_prf_2025`.
- Execute os comandos na ordem apresentada, especialmente os que criam views.
- Confira os nomes e os tipos das colunas após a importação.
- Leia a interpretação analítica depois de observar o resultado de cada consulta.

Convenções utilizadas

Elemento	Significado
<code>acidentes_prf_2025</code>	Tabela importada a partir do CSV da PRF.
<code>vw_acidentes_base</code>	View que acrescenta a variável <code>acidente_fatal</code> .
<code>acidente_fatal</code>	Indicador binário: 1 quando houve pelo menos uma morte; 0 caso contrário.
<code>perc_fatais</code>	Percentual de ocorrências classificadas como fatais.

1. Verificando a versão do SQLite

Contextualização

Antes de iniciar uma análise, é útil confirmar qual mecanismo de banco de dados está executando as consultas.

Comando SQL

```
SELECT sqlite_version() AS versao_sqlite;
```

Explicação linha por linha

SELECT sqlite_version() chama a função interna que informa a versão do SQLite.

AS versao_sqlite cria um nome mais legível para a coluna de saída.

Resultado esperado

Uma linha com a versão do mecanismo SQLite disponível no ambiente.

Interpretação

A versão ajuda a identificar quais funções e recursos podem ser utilizados.

Boa prática: No script original foi usado `SELECT version()`, que pertence ao DuckDB. No SQLite, utilize `sqlite_version()`.

2. Conferindo a estrutura da tabela

Contextualização

Depois de importar o CSV, o primeiro passo é verificar se as colunas foram reconhecidas corretamente.

Comando SQL

```
PRAGMA table_info(acidentes_prf_2025);
```

Explicação linha por linha

PRAGMA table_info solicita metadados sobre as colunas da tabela.

(acidentes_prf_2025) indica qual tabela será inspecionada.

Resultado esperado

Uma lista com posição, nome, tipo, nulabilidade, valor padrão e indicação de chave primária de cada coluna.

Interpretação

A inspeção permite identificar colunas numéricas importadas como texto e possíveis diferenças de nomenclatura.

Boa prática: O comando `DESCRIBE` usado no DuckDB é substituído por `PRAGMA table_info` no SQLite.

3. Contando o total de ocorrências

Contextualização

Uma contagem geral estabelece o tamanho da base e ajuda a validar se a importação foi concluída.

Comando SQL

```
SELECT COUNT(*) AS total_ocorrencias
FROM acidentes_prf_2025;
```

Explicação linha por linha

COUNT(*) conta todas as linhas da tabela, inclusive as que possuem valores nulos em alguma coluna.

AS total_ocorrencias renomeia o resultado.

FROM acidentes_prf_2025 define a fonte dos dados.

Resultado esperado

Uma única linha contendo o número total de registros importados.

Interpretação

Cada linha representa uma ocorrência de acidente no conjunto utilizado.

Boa prática: Compare a contagem com o total informado na fonte do arquivo para detectar falhas de importação.

4. Criando a variável acidente_fatal

Contextualização

Para analisar fatores associados a mortes, criamos uma variável binária que distingue ocorrências fatais e não fatais.

Comando SQL

```
DROP VIEW IF EXISTS vw_acidentes_base;

CREATE VIEW vw_acidentes_base AS
SELECT
    *,
    CASE
        WHEN CAST(mortos AS INTEGER) >= 1 THEN 1
        ELSE 0
    END AS acidente_fatal
FROM acidentes_prf_2025;
```

Explicação linha por linha

DROP VIEW IF EXISTS remove a view anterior somente quando ela já existe.

CREATE VIEW cria uma consulta reutilizável sem duplicar fisicamente os dados.

SELECT * preserva todas as colunas originais.

CAST(mortos AS INTEGER) converte a coluna mortos para número inteiro.

CASE ... END aplica uma regra condicional.

AS acidente_fatal atribui nome à nova variável.

Resultado esperado

A view passa a disponibilizar todas as colunas originais e a coluna acidente_fatal.

Interpretação

O valor 1 representa acidentes com uma ou mais mortes. O valor 0 representa os demais acidentes.

Boa prática: *CREATE OR REPLACE VIEW não é aceito pelo SQLite. Use DROP VIEW IF EXISTS seguido de CREATE VIEW.*

5. Calculando indicadores gerais

Contextualização

A análise começa com três medidas fundamentais: total de acidentes, quantidade de acidentes fatais e percentual de fatalidade.

Comando SQL

```
SELECT
    COUNT(*) AS total_acidentes,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base;
```

Explicação linha por linha

COUNT(*) calcula o total de ocorrências.

SUM(acidente_fatal) soma os valores 0 e 1; assim, o resultado equivale ao número de acidentes fatais.

100.0 * força o cálculo percentual com casas decimais.

ROUND(..., 2) arredonda o percentual para duas casas.

FROM vw_acidentes_base utiliza a view com a variável-alvo.

Resultado esperado

Uma linha com os três indicadores gerais.

Interpretação

O percentual de fatalidade deve ser interpretado como a proporção de ocorrências com ao menos uma morte, e não como a proporção de pessoas mortas.

Boa prática: Evite usar 100 sem decimal em cálculos de proporção; 100.0 deixa explícita a intenção de obter um resultado decimal.

6. Indicadores por unidade da Federação

Contextualização

Agrupar por UF permite comparar o volume de acidentes e a proporção de ocorrências fatais entre estados.

Comando SQL

```
SELECT
    uf,
    COUNT(*) AS total_acidentes,
    SUM(acidente_fatal) AS acidentes_fatais,
    SUM(CAST(mortos AS INTEGER)) AS total_mortos,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base
GROUP BY uf
HAVING COUNT(*) >= 100
ORDER BY perc_fatais DESC;
```

Explicação linha por linha

uf identifica a unidade da Federação.

GROUP BY uf forma um grupo para cada UF.

SUM(CAST(mortos AS INTEGER)) soma o número de mortos em cada estado.

HAVING COUNT(*) >= 100 mantém somente grupos com pelo menos 100 ocorrências.

ORDER BY perc_fatais DESC apresenta primeiro os maiores percentuais de fatalidade.

Resultado esperado

Uma linha por UF que atende ao limite mínimo de ocorrências.

Interpretação

O filtro mínimo reduz comparações instáveis causadas por grupos muito pequenos. Volume absoluto e percentual devem ser avaliados conjuntamente.

Boa prática: *WHERE* filtra linhas antes do agrupamento; *HAVING* filtra grupos depois do *GROUP BY*.

7. Ranking por rodovia federal

Contextualização

O ranking por BR identifica rodovias com maior número de mortos e permite priorizar investigações.

Comando SQL

```
SELECT
    br,
    COUNT(*) AS total_acidentes,
    SUM(CAST(mortos AS INTEGER)) AS total_mortos,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base
WHERE br IS NOT NULL
GROUP BY br
HAVING COUNT(*) >= 100
ORDER BY total_mortos DESC
LIMIT 30;
```

Explicação linha por linha

WHERE br IS NOT NULL exclui registros sem identificação de rodovia.

GROUP BY br agrupa as ocorrências por número da BR.

ORDER BY total_mortos DESC classifica pela quantidade total de mortos.

LIMIT 30 restringe a saída às trinta primeiras rodovias.

Resultado esperado

Até trinta linhas, uma para cada BR selecionada.

Interpretação

Uma rodovia pode aparecer com muitos mortos por ter grande fluxo e muitas ocorrências. A taxa de fatalidade oferece uma perspectiva complementar.

Boa prática: Defina claramente o critério do ranking: total de acidentes, total de mortos ou percentual de fatalidade respondem perguntas diferentes.

8. Construindo a série mensal

Contextualização

A análise temporal ajuda a reconhecer sazonalidade, mudanças e períodos de maior gravidade.

Comando SQL

```
SELECT
    CAST(strftime('%Y', data_inversa) AS INTEGER) AS ano,
    CAST(strftime('%m', data_inversa) AS INTEGER) AS mes,
    COUNT(*) AS total_acidentes,
    SUM(CAST(mortos AS INTEGER)) AS total_mortos,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base
GROUP BY ano, mes
ORDER BY ano, mes;
```

Explicação linha por linha

strftime('%Y', data_inversa) extrai o ano da data.

strftime('%m', data_inversa) extrai o mês.

CAST(... AS INTEGER) remove o formato textual e converte os componentes para números.

GROUP BY ano, mes consolida os registros por período mensal.

ORDER BY ano, mes organiza cronologicamente.

Resultado esperado

Uma linha por mês existente na base.

Interpretação

Compare simultaneamente volume, mortes e percentual de fatalidade. Um mês com muitos acidentes não é necessariamente o mês com maior proporção de ocorrências fatais.

Boa prática: *O SQLite usa strftime para extrair partes de datas. Isso substitui EXTRACT(YEAR/MONTH ...) do DuckDB.*

9. Análise bivariada por tipo de acidente

Contextualização

Esta consulta investiga quais tipos de acidente apresentam maior percentual de ocorrências fatais.

Comando SQL

```
SELECT
    tipo_acidente,
    COUNT(*) AS total_acidentes,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base
GROUP BY tipo_acidente
HAVING COUNT(*) >= 100
ORDER BY perc_fatais DESC;
```

Explicação linha por linha

tipo_acidente define a dimensão analisada: tipo de acidente.

COUNT(*) mede a frequência do grupo.

SUM(acidente_fatal) conta acidentes fatais.

perc_fatais compara a proporção de fatalidade entre categorias.

HAVING COUNT(*) >= 100 evita destacar categorias com base muito pequena.

Resultado esperado

Uma linha por categoria de tipo de acidente com pelo menos 100 registros.

Interpretação

A consulta mostra associação descritiva, não causalidade. Diferenças podem refletir exposição, localização, velocidade, tipo de veículo e outras variáveis não controladas.

Boa prática: Analise também valores ausentes, grafias diferentes e categorias raras antes de interpretar o ranking.

10. Análise bivariada por causa do acidente

Contextualização

Esta consulta investiga quais causas declaradas estão associadas a maior fatalidade.

Comando SQL

```
SELECT
    causa_acidente,
    COUNT(*) AS total_acidentes,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base
GROUP BY causa_acidente
HAVING COUNT(*) >= 100
ORDER BY perc_fatais DESC
LIMIT 30;
```

Explicação linha por linha

causa_acidente define a dimensão analisada: causa do acidente.

COUNT(*) mede a frequência do grupo.

SUM(acidente_fatal) conta acidentes fatais.

perc_fatais compara a proporção de fatalidade entre categorias.

HAVING COUNT(*) >= 100 evita destacar categorias com base muito pequena.

Resultado esperado

Uma linha por categoria de causa do acidente com pelo menos 100 registros.

Interpretação

A consulta mostra associação descritiva, não causalidade. Diferenças podem refletir exposição, localização, velocidade, tipo de veículo e outras variáveis não controladas.

Boa prática: Analise também valores ausentes, grafias diferentes e categorias raras antes de interpretar o ranking.

11. Análise bivariada por fase do dia

Contextualização

Esta consulta investiga como a luminosidade e o período do dia se relacionam com a gravidade.

Comando SQL

```
SELECT
    fase_dia,
    COUNT(*) AS total_acidentes,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base
GROUP BY fase_dia
HAVING COUNT(*) >= 100
ORDER BY perc_fatais DESC;
```

Explicação linha por linha

fase_dia define a dimensão analisada: fase do dia.

COUNT(*) mede a frequência do grupo.

SUM(acidente_fatal) conta acidentes fatais.

perc_fatais compara a proporção de fatalidade entre categorias.

HAVING COUNT(*) >= 100 evita destacar categorias com base muito pequena.

Resultado esperado

Uma linha por categoria de fase do dia com pelo menos 100 registros.

Interpretação

A consulta mostra associação descritiva, não causalidade. Diferenças podem refletir exposição, localização, velocidade, tipo de veículo e outras variáveis não controladas.

Boa prática: Analise também valores ausentes, grafias diferentes e categorias raras antes de interpretar o ranking.

12. Análise bivariada por condição meteorológica

Contextualização

Esta consulta investiga como diferentes condições meteorológicas se distribuem entre ocorrências fatais.

Comando SQL

```
SELECT
    condicao_meteorologica,
    COUNT(*) AS total_acidentes,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
```

```
FROM vw_acidentes_base
GROUP BY condicao_meteorologica
HAVING COUNT(*) >= 100
ORDER BY perc_fatais DESC;
```

Explicação linha por linha

condicao_meteorologica define a dimensão analisada: condição meteorológica.

COUNT(*) mede a frequência do grupo.

SUM(acidente_fatal) conta acidentes fatais.

perc_fatais compara a proporção de fatalidade entre categorias.

HAVING COUNT(*) >= 100 evita destacar categorias com base muito pequena.

Resultado esperado

Uma linha por categoria de condição meteorológica com pelo menos 100 registros.

Interpretação

A consulta mostra associação descritiva, não causalidade. Diferenças podem refletir exposição, localização, velocidade, tipo de veículo e outras variáveis não controladas.

Boa prática: Analise também valores ausentes, grafias diferentes e categorias raras antes de interpretar o ranking.

13. Análise bivariada por tipo de pista

Contextualização

Esta consulta investiga se pistas simples, duplas ou múltiplas apresentam diferenças na proporção de fatalidade.

Comando SQL

```
SELECT
    tipo_pista,
    COUNT(*) AS total_acidentes,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base
GROUP BY tipo_pista
HAVING COUNT(*) >= 100
ORDER BY perc_fatais DESC;
```

Explicação linha por linha

tipo_pista define a dimensão analisada: tipo de pista.

COUNT(*) mede a frequência do grupo.

SUM(acidente_fatal) conta acidentes fatais.

perc_fatais compara a proporção de fatalidade entre categorias.

HAVING COUNT(*) >= 100 evita destacar categorias com base muito pequena.

Resultado esperado

Uma linha por categoria de tipo de pista com pelo menos 100 registros.

Interpretação

A consulta mostra associação descritiva, não causalidade. Diferenças podem refletir exposição, localização, velocidade, tipo de veículo e outras variáveis não controladas.

Boa prática: Analise também valores ausentes, grafias diferentes e categorias raras antes de interpretar o ranking.

14. Combinando tipo de pista e fase do dia

Contextualização

Combinar duas variáveis permite identificar contextos específicos de maior risco, como determinado tipo de pista em determinado período.

Comando SQL

```
SELECT
    tipo_pista,
    fase_dia,
    COUNT(*) AS total_acidentes,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS cobertura_perc,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base
GROUP BY tipo_pista, fase_dia
HAVING COUNT(*) >= 100
ORDER BY perc_fatais DESC;
```

Explicação linha por linha

GROUP BY tipo_pista, fase_dia cria grupos para cada combinação.

SUM(COUNT(*)) OVER () calcula o total somando as contagens dos grupos.

cobertura_perc mostra a participação de cada combinação no conjunto agrupado.

perc_fatais mede a proporção de ocorrências fatais na combinação.

Resultado esperado

Uma linha por combinação com frequência mínima de 100 registros.

Interpretação

Cobertura alta indica um contexto frequente; fatalidade alta indica maior gravidade proporcional. Prioridades podem considerar as duas medidas.

Boa prática: Funções de janela dependem da versão do SQLite. Em versões modernas, `OVER()` é suportado.

15. Calculando confiança e lift por tipo de acidente

Contextualização

O lift compara a taxa de fatalidade de cada tipo de acidente com a taxa geral da base.

Comando SQL

```
WITH taxa_global AS (  
    SELECT 1.0 * SUM(acidente_fatal) / COUNT(*) AS taxa  
    FROM vw_acidentes_base  
)  
SELECT  
    tipo_acidente,  
    COUNT(*) AS total_acidentes,  
    SUM(acidente_fatal) AS acidentes_fatais,  
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS cobertura_perc,  
    ROUND(1.0 * SUM(acidente_fatal) / COUNT(*), 4) AS confianca,  
    ROUND((1.0 * SUM(acidente_fatal) / COUNT(*)) / taxa, 2) AS lift  
FROM vw_acidentes_base  
CROSS JOIN taxa_global  
GROUP BY tipo_acidente, taxa  
HAVING COUNT(*) >= 100  
ORDER BY lift DESC;
```

Explicação linha por linha

WITH taxa_global AS (...) cria uma CTE com a taxa geral de fatalidade.

CROSS JOIN taxa_global disponibiliza a taxa global para cada grupo.

confianca representa a taxa de fatalidade dentro do tipo de acidente.

lift divide a taxa do grupo pela taxa geral.

ORDER BY lift DESC coloca primeiro os grupos mais acima da referência global.

Resultado esperado

Uma linha por tipo de acidente, com cobertura, confiança e lift.

Interpretação

Lift maior que 1 indica taxa de fatalidade superior à taxa global; igual a 1 indica taxa semelhante; menor que 1 indica taxa inferior. O lift não prova relação causal.

Boa prática: Não interprete lift isoladamente. Verifique também cobertura e quantidade absoluta de casos.

16. Criando uma view de indicadores mensais

Contextualização

Uma view mensal organiza a lógica de agregação para reutilização em relatórios e dashboards.

Comando SQL

```
DROP VIEW IF EXISTS vw_indicadores_mensais;

CREATE VIEW vw_indicadores_mensais AS
SELECT
    CAST(strftime('%Y', data_inversa) AS INTEGER) AS ano,
    CAST(strftime('%m', data_inversa) AS INTEGER) AS mes,
    COUNT(*) AS total_acidentes,
    SUM(CAST(mortos AS INTEGER)) AS total_mortos,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base
GROUP BY ano, mes;

SELECT *
FROM vw_indicadores_mensais
ORDER BY ano, mes;
```

Explicação linha por linha

DROP VIEW IF EXISTS permite recriar a view sem erro.

CREATE VIEW armazena a definição da consulta.

GROUP BY ano, mes gera os indicadores mensais.

SELECT * FROM vw_indicadores_mensais consulta a view.

ORDER BY é aplicado na consulta final para garantir a ordenação.

Resultado esperado

A view fica disponível no banco e retorna indicadores consolidados por mês.

Interpretação

Views formam uma camada semântica: nomes e regras de negócio ficam centralizados e podem ser reutilizados.

Boa prática: Não dependa de *ORDER BY* dentro da definição da view. Ordene sempre na consulta que consome a view.

17. Criando uma view por UF e BR

Contextualização

A combinação UF-BR permite analisar a mesma rodovia separadamente em cada estado.

Comando SQL

```
DROP VIEW IF EXISTS vw_indicadores_uf_br;

CREATE VIEW vw_indicadores_uf_br AS
SELECT
    uf,
    br,
    COUNT(*) AS total_acidentes,
    SUM(CAST(mortos AS INTEGER)) AS total_mortos,
    SUM(acidente_fatal) AS acidentes_fatais,
    ROUND(100.0 * SUM(acidente_fatal) / COUNT(*), 2) AS perc_fatais
FROM vw_acidentes_base
WHERE br IS NOT NULL
GROUP BY uf, br;

SELECT *
FROM vw_indicadores_uf_br
ORDER BY total_mortos DESC;
```

Explicação linha por linha

uf, br definem a granularidade da análise.

WHERE br IS NOT NULL exclui ocorrências sem identificação da BR.

GROUP BY uf, br gera uma linha para cada combinação.

ORDER BY total_mortos DESC cria um ranking na consulta final.

Resultado esperado

Uma tabela lógica de indicadores por trecho estadual de rodovia.

Interpretação

A mesma BR pode apresentar padrões distintos em estados diferentes devido a infraestrutura, extensão, relevo, fluxo e fiscalização.

Boa prática: *Declare explicitamente a granularidade de cada indicador para evitar comparações indevidas.*

18. Construindo uma base analítica

Contextualização

Uma base analítica seleciona apenas os campos necessários para análise, visualização ou modelagem posterior.

Comando SQL

```
DROP VIEW IF EXISTS vw_base_analitica;

CREATE VIEW vw_base_analitica AS
SELECT
    data_inversa,
    dia_semana,
    horario,
```

```

    uf,
    br,
    municipio,
    causa_acidente,
    tipo_acidente,
    classificacao_acidente,
    fase_dia,
    condicao_meteorologica,
    tipo_pista,
    tracado_via,
    uso_solo,
    CAST(mortos AS INTEGER) AS mortos,
    acidente_fatal
FROM vw_acidentes_base;

SELECT *
FROM vw_base_analitica
LIMIT 20;

```

Explicação linha por linha

CREATE VIEW vw_base_analitica cria uma camada organizada para consumo.

lista de colunas restringe a saída às variáveis úteis.

CAST(mortos AS INTEGER) padroniza o tipo numérico.

LIMIT 20 permite inspecionar uma amostra sem retornar toda a base.

Resultado esperado

Uma view com variáveis temporais, espaciais, contextuais e a variável-alvo.

Interpretação

A base pode ser exportada pela interface do SQLiteOnline e utilizada em ferramentas de BI, planilhas ou linguagens de programação.

Boa prática: Evite *SELECT ** em bases finais. Listar colunas torna a estrutura explícita e reduz surpresas quando a fonte muda.

19. Construindo uma base preliminar para modelagem

Contextualização

Para projetos preditivos, é comum selecionar variáveis explicativas e uma variável-alvo em uma tabela limpa e consistente.

Comando SQL

```

DROP VIEW IF EXISTS vw_base_modelavel_preliminar;

CREATE VIEW vw_base_modelavel_preliminar AS
SELECT
    uf,
    br,

```

```
municipio,  
CAST(strftime('%m', data_inversa) AS INTEGER) AS mes,  
dia_semana,  
fase_dia,  
causa_acidente,  
tipo_acidente,  
condicao_meteorologica,  
tipo_pista,  
tracado_via,  
uso_solo,  
acidente_fatal  
FROM vw_acidentes_base;  
  
SELECT *  
FROM vw_base_modelavel_preliminar  
LIMIT 20;
```

Explicação linha por linha

mes transforma a data em uma variável temporal simples.

variáveis categóricas descrevem localização e contexto da ocorrência.

acidente_fatal funciona como variável-alvo binária.

LIMIT 20 inspeciona os primeiros registros.

Resultado esperado

Uma view preliminar pronta para avaliação de qualidade e transformação posterior.

Interpretação

Antes de modelar, ainda será necessário tratar valores ausentes, categorias raras, codificação de variáveis e separação entre treino e teste.

Boa prática: *Uma base modelável não é automaticamente um modelo. SQL prepara e organiza os dados; a etapa estatística exige validação própria.*

Apêndice A — Diferenças entre DuckDB e SQLiteOnline

No DuckDB	No SQLiteOnline	Motivo
read_csv_auto(...)	Importação pela interface	SQLite não possui função SQL nativa equivalente no ambiente online.
CREATE OR REPLACE TABLE	Importar a tabela ou usar DROP TABLE + CREATE TABLE	SQLite não implementa CREATE OR REPLACE TABLE.
CREATE OR REPLACE VIEW	DROP VIEW IF EXISTS + CREATE VIEW	É a forma compatível de recriar views.
DESCRIBE tabela	PRAGMA table_info(tabela)	PRAGMA fornece os metadados das colunas.
EXTRACT(YEAR/MONTH FROM data)	strftime('%Y'/'%m', data)	SQLite utiliza funções próprias para datas.
COPY ... TO CSV	Exportação pela interface	A gravação de arquivos depende do ambiente hospedeiro.

Apêndice B — Guia rápido dos comandos

Comando	Finalidade
SELECT	Seleciona colunas e expressões.
FROM	Define a tabela ou view consultada.
WHERE	Filtra linhas antes do agrupamento.
GROUP BY	Agrupar linhas para cálculo de indicadores.
HAVING	Filtra grupos após a agregação.
ORDER BY	Ordena o resultado.
LIMIT	Limita a quantidade de linhas.
COUNT	Conta registros.
SUM	Soma valores.
ROUND	Arredonda números.
CAST	Converte tipos.
CASE	Cria regras condicionais.
WITH	Cria uma expressão de tabela comum.
OVER	Define uma função de janela.
CREATE VIEW	Cria uma consulta reutilizável.
DROP VIEW IF EXISTS	Remove uma view apenas quando ela existe.
PRAGMA table_info	Mostra a estrutura de uma tabela.
strftime	Extraí ou formata partes de datas.

Apêndice C — Exercícios propostos

1. Conte o total de acidentes por município e apresente os vinte municípios com maior número de ocorrências.
2. Calcule o total de mortos e o percentual de fatalidade por dia da semana.
3. Compare acidentes fatais entre áreas urbanas e rurais usando a coluna `uso_solo`.
4. Crie uma consulta que mostre apenas UFs com percentual de fatalidade superior à taxa global.
5. Construa uma view com indicadores por município e BR.
6. Calcule cobertura, confiança e lift para `causa_acidente`.
7. Crie uma série mensal separada por UF.
8. Identifique combinações de condição meteorológica e fase do dia com pelo menos 100 ocorrências.
9. Crie uma base analítica contendo somente acidentes fatais.
10. Explique por que uma associação observada em SQL não permite concluir causalidade.